

Interview Project Guide: MLOps Drift Monitor and Automated Retraining Platform

Use this guide as a speaking companion for interviews. It is written so you can quickly pull out concise answers for technical deep-dives, product discussions, and behavioral questions.

30-Second Pitch

This project is an end-to-end MLOps platform for monitoring a machine learning model after deployment. It serves fraud predictions through a FastAPI API, logs every inference to SQLite, continuously checks for data drift, prediction drift, and concept drift, raises alerts when model behavior changes, and can trigger an automated retraining pipeline that evaluates and promotes a better model through MLflow.

The core problem it solves is silent model degradation. A model can look good at training time, but production data changes. This system closes that feedback loop by combining prediction serving, telemetry, drift detection, alerting, retraining, model registry management, and a live dashboard in one cohesive workflow.

Project Overview

What The Project Is

The project is a production-style ML lifecycle monitoring platform built around a credit card fraud detection use case. It demonstrates how a trained model can be served, observed, evaluated, retrained, and redeployed without treating model training as a one-time event.

The system includes:

- A FastAPI prediction service for single and batch inference.
- A SQLite telemetry store for predictions, labels, drift results, alerts, model versions, and retraining events.
- Drift detectors for feature distribution shift, prediction distribution shift, and performance degradation.
- A rule-based retraining decision engine.
- An MLflow-backed model training, tracking, registry, and promotion workflow.
- A Plotly Dash dashboard for monitoring health, drift, alerts, and model registry state.
- Scripts for baseline training, production traffic simulation, drift injection, and full demo orchestration.

The Problem It Solves

The project solves the gap between "I trained a good model" and "I know this model is still reliable

in production."

In real systems, model quality can degrade because:

- User behavior changes.
- Input distributions shift.
- Upstream data pipelines change.
- Fraud patterns evolve.
- Ground-truth labels arrive late.
- Confidence distributions drift before accuracy visibly fails.

This platform monitors those risks through multiple signals:

- Data drift using Population Stability Index and KL divergence.
- Prediction drift using Hellinger distance and output distribution statistics.
- Concept drift using drops in accuracy, F1, precision, and recall once true labels are available.

Who Benefits And Why It Matters

Data science teams benefit because they can see when a model trained on historical data no longer matches current reality.

ML engineers benefit because the project demonstrates serving, logging, scheduling, registry integration, and deployment control in one system.

Business stakeholders benefit because drift alerts reduce the time between performance degradation and corrective action.

Risk, fraud, and operations teams benefit because they get visibility into whether the fraud model is stable, degraded, or in need of retraining.

In interview language:

"The project matters because production ML is not only about training. It is about observability, feedback loops, and controlled model lifecycle management."

Technical Foundation

Complete Tech Stack

Layer	Technology	Role
Language	Python 3.11+	Core implementation language
API	FastAPI	Prediction, health, monitoring, model, and feedback endpoints
ASGI Server	Uvicorn	Runs the FastAPI app locally or

Layer	Technology	Role
		in deployment
ML Framework	scikit-learn	RandomForest model, preprocessing, metrics
Data Processing	pandas, NumPy	Dataset loading, feature arrays, statistics, drift calculations
Experiment Tracking	MLflow	Tracks model runs, artifacts, metrics, and registry versions
Dashboard	Plotly Dash	Interactive monitoring UI
Charts	Plotly	Drift trends, heatmaps, confusion matrix, ROC, model comparisons
UI Components	dash-bootstrap-components	Responsive layout and cards
Database	SQLite with SQLAlchemy	Lightweight local persistence and ORM models
Configuration	Pydantic Settings, YAML, dotenv	Typed settings, thresholds, alert configs, environment overrides
Scheduler	APScheduler	Periodic drift checks in the FastAPI process
HTTP Clients	requests, httpx	Demo traffic, tests, endpoint calls
Serialization	joblib, JSON	Model preprocessing artifacts and logged feature payloads
Testing	pytest, unittest, FastAPI TestClient	Unit and integration-style validation
Alerting	Console, Slack webhook, SMTP email	Notification channels for warning and critical drift

Why These Technologies Were Chosen

Python

Python is the natural fit because the project is ML-heavy and relies on mature data science and MLOps libraries. It provides strong ecosystem support for model training, evaluation, APIs, dashboards, and scripting.

Alternative considered: Java or Go would be stronger for highly concurrent services, but they would make ML experimentation and statistical workflows more cumbersome.

FastAPI

FastAPI was chosen because it is lightweight, fast, type-friendly, and integrates naturally with Pydantic schemas. It also gives automatic OpenAPI documentation, which is useful for explaining and testing prediction endpoints.

Alternative considered: Flask is simpler but has weaker built-in validation and API schema support. Django would be too heavy for a model serving microservice.

SQLite and SQLAlchemy

SQLite was chosen for local reproducibility and interview/demo friendliness. It allows the project to run without provisioning Postgres, MySQL, or a cloud database. SQLAlchemy provides a clean persistence abstraction and ORM models.

Alternative considered: Postgres would be better for production concurrency and multi-service deployments, but SQLite keeps the demo portable and easy to run.

MLflow

MLflow was chosen because it is a standard tool for experiment tracking and model registry workflows. It helps show that the project is not just retraining a model, but tracking lineage, metrics, artifacts, versions, and promotion status.

Alternative considered: Weights & Biases or Neptune could work for tracking, but MLflow is open-source, local-friendly, and widely recognized in MLOps interviews.

scikit-learn

scikit-learn was chosen because the model task is tabular fraud classification. RandomForest provides a strong baseline, handles nonlinear relationships, and is easy to serialize, evaluate, and explain.

Alternative considered: XGBoost or LightGBM could improve performance, but RandomForest is simpler and keeps the focus on MLOps lifecycle rather than model competition.

Plotly Dash

Dash was chosen because it lets the project build a Python-native monitoring dashboard without introducing a separate frontend framework. Plotly handles rich charting for drift and performance metrics.

Alternative considered: React would offer more UI control, but would increase project complexity. Streamlit would be faster to prototype but less suitable for a multi-page operational dashboard.

APScheduler

APScheduler was chosen to run periodic drift checks inside the API process for the demo. It is simple and keeps the project self-contained.

Alternative considered: Airflow, Prefect, or Celery would be stronger for production orchestration, but are heavier than needed for a portable local project.

How The Technologies Work Together

The serving layer receives a transaction through FastAPI. Pydantic validates the request schema. The active model is loaded through an in-memory ModelProvider cache and uses the saved preprocessor to transform the input. The prediction result is logged into SQLite through SQLAlchemy.

APScheduler periodically asks the DriftManager to load a sliding window of recent predictions from SQLite. The DriftManager sends feature arrays, prediction labels, confidence scores, and available ground-truth labels to specialized detectors. Those detectors compute PSI, KL divergence, Hellinger distance, and metric degradation. Results are stored back into SQLite.

The RetrainingDecisionEngine evaluates drift severity against YAML-configured thresholds. If the rules trigger retraining, the RetrainingPipeline trains a candidate model, logs it to MLflow, compares it with the current production model, and deploys it if it improves. The dashboard reads from SQLite and visualizes the operational state.

Interview quote:

"I designed the system around a feedback loop: serve, log, monitor, decide, retrain, evaluate, deploy, and continue observing."

Code Architecture

High-Level System Design

The code is organized by responsibility:

- `src/api`: FastAPI app factory, middleware, schemas, and route modules.
- `src/data`: database models, prediction logging, dataset loading, and drift injection helpers.
- `src/monitoring`: data drift, prediction drift, concept drift, and DriftManager orchestration.
- `src/decision`: retraining decision rules.
- `src/models`: model training, evaluation, and MLflow registry wrapper.
- `src/pipeline`: retraining pipeline and model deployer.
- `src/alerting`: console, Slack, and email notification managers.
- `src/dashboard`: Dash app, layouts, components, and callbacks.
- `scripts`: operational workflows for setup, training, simulation, drift injection, and demo orchestration.
- `tests`: unit and integration-style tests for critical behavior.

Main Data Flow

1. Client sends transaction features to `/predict`.
2. FastAPI validates request through Pydantic.
3. ModelProvider returns the currently active model and preprocessor.
4. Model generates prediction and confidence score.

5. PredictionLogger writes the input, output, latency, model version, and timestamp to SQLite.
6. Ground truth can later be submitted through `/ground-truth/{prediction_id}`.
7. Scheduler or manual `/drift/run` triggers DriftManager.
8. DriftManager computes drift across the recent prediction window.
9. Drift results and alerts are persisted.
10. RetrainingDecisionEngine decides whether retraining is needed.
11. RetrainingPipeline trains and evaluates a candidate model.
12. ModelDeployer promotes and hot-swaps the active model if the candidate wins.
13. Dash reads persisted state and visualizes the system.

Key Architectural Decisions

Separate Detectors For Different Drift Types

Data drift, prediction drift, and concept drift are implemented as separate components. This keeps the logic easier to test and explain.

Why it matters:

- Data drift can happen before labels arrive.
- Prediction drift can show output behavior changing even if features look similar.
- Concept drift requires ground truth and directly measures model performance degradation.

Interview quote:

"I separated drift into covariate shift, output distribution shift, and performance degradation because each signal answers a different operational question."

App Factory Pattern

FastAPI uses a `create_app()` factory and a lifespan manager. This keeps startup concerns centralized: configuration loading, database initialization, model loading, drift detector setup, scheduler startup, and shutdown cleanup.

Why it matters:

- Easier testing with FastAPI TestClient.
- Cleaner initialization order.
- Better separation between app creation and runtime behavior.

ModelProvider Cache

The active production model is stored in a centralized in-memory provider. Prediction endpoints do not repeatedly load the model from disk or MLflow.

Why it matters:

- Lower latency during inference.
- Simpler hot-swap deployment.
- Clear ownership of the active model version.

SQLite For Demo Persistence

The project uses SQLite as an embedded operational database. It stores telemetry and audit trails without external services.

Why it matters:

- Easy to run in interviews and local demos.
- Makes the full system portable.
- Demonstrates schema design without infrastructure overhead.

Production note:

"For production, I would replace SQLite with Postgres and likely move scheduling/retraining to a separate worker."

Rule-Based Retraining Engine

The retraining decision logic is rule-based instead of fully automated by a learned controller.

Why it matters:

- Easy to explain.
- Auditable.
- Safer for high-impact workflows.
- Thresholds can be tuned by domain owners.

MLflow Registry Mirror

The project uses MLflow for model lifecycle tracking and also stores model metadata in SQLite. This gives the dashboard fast access to production status and metric history.

Why it matters:

- MLflow is the source of model artifacts and experiment lineage.
- SQLite gives the application fast operational queries for UI and auditing.

Design Patterns Used

Factory pattern:

- `create_app( )` builds the FastAPI app.
- `create_dash_app( )` builds the dashboard app.

Repository/service-style pattern:

- Database access is wrapped through managers and loggers instead of being scattered across

endpoints.

Strategy-like separation:

- DataDriftDetector, PredictionDriftDetector, and ConceptDriftDetector each implement a specific detection strategy.

Facade/orchestrator pattern:

- DriftManager hides the complexity of running multiple detectors and persisting results.
- RetrainingPipeline hides the complexity of training, evaluation, registry, and deployment.

Dependency injection:

- FastAPI dependencies provide database and logger instances.

Configuration-driven design:

- YAML and Pydantic settings allow thresholds, model parameters, dashboard refresh intervals, alert channels, and database settings to be changed without editing core code.

Scalability And Maintainability Considerations

The current architecture is intentionally local and demo-friendly, but it is organized so it can evolve.

Scalability path:

- Replace SQLite with Postgres.
- Move APScheduler jobs to Airflow, Prefect, or Celery workers.
- Containerize API, dashboard, MLflow, and database separately.
- Add Prometheus/Grafana for infrastructure metrics.
- Store feature/reference data in object storage.
- Add a queue for prediction logging under high throughput.
- Split retraining into asynchronous jobs with retry handling.

Maintainability strengths:

- Clear module boundaries.
- Testable detector and decision logic.
- Configurable thresholds.
- Centralized database models.
- Separate dashboard layouts and callbacks.
- Operational scripts for repeatable demos.

Potential maintainability risks:

- Running scheduler inside the API process is acceptable for local use but not ideal for distributed production.

- SQLite has concurrency limits.
 - Dashboard callbacks directly query the database; for production, a service/API layer or materialized metrics table would be cleaner.
-

Project Benefits

Business Advantages

- Reduces the risk of undetected model degradation.
- Improves trust in ML systems by making model health visible.
- Shortens the time from drift detection to corrective action.
- Provides audit trails for predictions, drift checks, alerts, retraining, and model promotion.
- Helps teams explain why a model was retrained and whether the candidate improved.

Technical Advantages

- End-to-end MLOps workflow instead of isolated scripts.
- Multiple drift detection methods instead of relying on one metric.
- Supports late-arriving labels for concept drift.
- Uses MLflow for model tracking and registry operations.
- Keeps serving latency low through in-memory model caching.
- Uses typed API schemas for request validation.
- Uses modular detectors and rule engines that can be individually tested.

Efficiency Gains

- Automated telemetry logging removes manual monitoring.
- Scheduled drift checks remove the need for ad hoc analysis.
- Automated retraining reduces manual recovery time.
- Dashboard reduces debugging time by centralizing model health signals.
- Demo scripts make it easy to reproduce normal and drifted scenarios.

User Experience Enhancements

The dashboard gives a user-friendly way to understand:

- Total predictions and throughput.
- Current PSI and drifted features.
- Hellinger distance and prediction distribution shift.
- F1, precision, recall, accuracy, confusion matrix, and ROC behavior.

- Alert history and severity.
- Production model version and retraining history.

In product-focused interviews:

"The dashboard turns model health from a hidden backend concern into something operators can see, explain, and act on."

Long-Term Value And Extensibility

The project can be extended in several directions:

- Support additional model types like XGBoost, LightGBM, or neural networks.
 - Add Evidently or Alibi Detect for richer drift reports.
 - Add Prometheus metrics for production observability.
 - Support online feature stores.
 - Add role-based dashboard access.
 - Add model approval gates before deployment.
 - Add shadow deployment or canary release logic.
 - Add data quality checks before drift checks.
 - Add retraining cost and fairness monitoring.
-

Interview-Ready Context

Key Metrics And Achievements

Use only metrics you can defend from the code and docs:

- Implements a full closed-loop MLOps lifecycle: serve, log, monitor, alert, retrain, evaluate, deploy.
- Covers three drift families: data drift, prediction drift, and concept drift.
- Tracks multiple model metrics: accuracy, F1, precision, recall, AUC-ROC, confusion matrix, and ROC curve.
- Provides API endpoints for prediction, batch prediction, ground-truth feedback, model info, drift checks, alerts, and health.
- Includes automated tests for drift math, data logging, prediction API, decision engine, and retraining orchestration.
- Includes operational scripts for reproducible demo, drift injection, baseline training, and traffic simulation.

If asked about scale:

"This version is optimized for local reproducibility and architectural completeness. The next step for production scale would be replacing SQLite with Postgres, moving retraining into a worker queue, and containerizing services."

Challenges Overcome

Challenge: Monitoring Without Immediate Labels

Ground-truth labels often arrive late in real ML systems. The solution was to monitor three layers:

- Data drift, which does not require labels.
- Prediction drift, which uses model outputs and confidence scores.
- Concept drift, which activates when true labels arrive.

Interview framing:

"I did not want the system to wait for labels before detecting risk, so I added unlabeled drift signals as early warning indicators."

Challenge: Avoiding A Monolithic ML Script

The project could have been a single training script, but that would not represent production ML. The solution was to separate API serving, drift detection, alerting, retraining, registry, deployment, and dashboard concerns.

Interview framing:

"I treated this as an operational platform, not a notebook converted into scripts."

Challenge: Making Retraining Safe

Automatically retraining is risky if the candidate model is worse. The solution was a champion/challenger comparison before deployment.

Interview framing:

"Retraining does not automatically mean deployment. The pipeline compares candidate metrics against the current production model before promotion."

Challenge: Demo Reproducibility

MLOps systems often require many services. The solution was to keep the local stack lightweight with SQLite, local MLflow, and scripts that orchestrate training, traffic, drift, API, and dashboard.

Interview framing:

"I intentionally optimized the first version for reproducibility, because a project that cannot be run and demonstrated is hard to evaluate."

What I Would Do Differently With Hindsight

If this were a production system, I would make several changes:

- Use Postgres instead of SQLite for concurrency, durability, and multi-process access.

- Move scheduled drift checks and retraining out of the API process into a worker service.
- Use a message queue like Kafka, RabbitMQ, or Redis Streams for prediction telemetry.
- Add Docker Compose or Kubernetes manifests for repeatable deployment.
- Add model approval gates or canary deployment before full promotion.
- Add a proper feature store or online/offline feature consistency checks.
- Add authentication and authorization for the dashboard and API.
- Add Prometheus metrics and structured observability beyond application logs.
- Add data quality checks for schema drift, null rates, range violations, and feature freshness.

Good interview phrasing:

"The current version demonstrates the MLOps control loop. For production, I would separate compute concerns and introduce more durable infrastructure around database, scheduling, queues, and deployment safety."

Future Roadmap

Short-term:

- Add Docker Compose for API, dashboard, MLflow, and database.
- Add CI workflow for tests and linting.
- Add screenshots or a GIF to the README.
- Add endpoint examples for all API routes.
- Add richer drift explanations in the dashboard.

Medium-term:

- Replace SQLite with Postgres.
- Add asynchronous workers for retraining.
- Add Prometheus and Grafana integration.
- Add alert deduplication tuning and alert severity routing.
- Add support for multiple models and multiple datasets.

Long-term:

- Add canary deployments.
- Add model approval workflows.
- Add feature store integration.
- Add fairness monitoring.
- Add data contracts and schema validation.
- Add automated rollback if post-deployment health worsens.

Answers For Common Interview Questions

"Explain Your Project In Simple Terms."

It is a monitoring and retraining platform for machine learning models. It serves predictions, records what the model saw and predicted, checks whether the data or model behavior is drifting, alerts the team when risk increases, and can retrain and promote a new model if performance degrades.

"What Makes This An MLOps Project?"

It goes beyond model training. It includes serving, telemetry, drift detection, scheduling, alerting, model registry integration, automated retraining, deployment control, and monitoring UI. Those are the operational pieces needed to keep ML reliable after deployment.

"Why Did You Use Multiple Drift Metrics?"

Because drift is not one thing. PSI and KL detect feature distribution changes. Hellinger distance detects prediction distribution changes. Performance metrics detect concept drift when labels are available. Together they give earlier and more reliable warning than one metric alone.

"How Does Retraining Work?"

The system runs drift checks over recent predictions. If thresholds are breached, the decision engine evaluates severity. For high-risk degradation, the retraining pipeline loads data, trains a candidate model, logs it to MLflow, compares it with the current champion, and deploys it only if it improves.

"How Do You Prevent Bad Models From Being Deployed?"

The pipeline uses a champion/challenger evaluation. The current production model is the champion. The newly trained model is the challenger. The challenger must improve according to selected metrics before the deployment step promotes it.

"Why SQLite?"

SQLite makes the project easy to run locally and demonstrate without external infrastructure. It is enough for a single-node demo and stores telemetry, drift results, alerts, model versions, and retraining events. For production, I would move to Postgres.

"Why FastAPI?"

FastAPI gives typed request validation, automatic OpenAPI docs, strong performance, and clean dependency injection. It is a good fit for ML serving because schemas matter: every prediction request must have the expected features.

"Why MLflow?"

MLflow provides model lineage: parameters, metrics, artifacts, model versions, and registry status. That matters because retraining without tracking creates reproducibility and governance problems.

"How Would You Scale This?"

I would separate services: API, dashboard, scheduler, retraining worker, database, and MLflow tracking server. I would use Postgres for state, a queue for telemetry or retraining jobs, object storage for artifacts, and container orchestration for deployment.

"What Was The Hardest Part?"

The hardest part was designing the feedback loop so it was realistic but still demo-friendly. I had to balance production concepts like drift monitoring, retraining, and model promotion with a local stack that can run without cloud infrastructure.

"What Are The Limitations?"

The current system is local-first. SQLite is not ideal for high concurrency, the scheduler runs inside the API process, and deployment is an in-memory hot-swap rather than a multi-instance rollout. Those are intentional simplifications for a portable MLOps project.

Interview Extraction Notes

For Technical Deep-Dives

Focus on:

- Drift metric choices.
- FastAPI lifecycle and ModelProvider cache.
- DriftManager orchestration.
- RetrainingDecisionEngine rules.
- MLflow registry and champion/challenger comparison.
- SQLite schema and persistence model.
- Production scaling path.

Strong quote:

"The key architectural choice was separating serving, monitoring, decisioning, and retraining so each part can evolve independently."

For Behavioral Interviews

Focus on:

- You identified silent model degradation as the real-world problem.
- You broke a complex system into modules.
- You made tradeoffs for reproducibility.
- You planned production improvements honestly.

- You built for observability and accountability.

Strong quote:

"I learned that a model's real value depends on the system around it: monitoring, feedback, retraining, and controlled deployment."

For Product-Focused Interviews

Focus on:

- Business risk of stale fraud models.
- Dashboard visibility for operators.
- Alerts reducing time-to-detection.
- Automated retraining reducing time-to-recovery.
- Audit trails supporting accountability.

Strong quote:

"The product value is that teams no longer have to guess whether the model is healthy. They can see degradation, understand why it happened, and act quickly."

For Resume Or Portfolio Discussion

Use this compact bullet:

Built an end-to-end MLOps drift monitoring platform for fraud detection using FastAPI, scikit-learn, MLflow, SQLite, APScheduler, and Plotly Dash. The system serves predictions, logs inference telemetry, detects data/prediction/concept drift, raises alerts, triggers automated retraining, compares candidate models, promotes improved versions, and visualizes model health in a live dashboard.

Final Defense Statement

If an interviewer challenges whether this is "production-grade," answer with nuance:

"I would call it production-style rather than production-complete. It implements the core control loop you need in production ML: serving, telemetry, drift detection, alerting, retraining, evaluation, and deployment. The infrastructure choices are intentionally lightweight for local reproducibility. To make it production-grade, I would externalize scheduling and retraining, move persistence to Postgres, add queues, containerize the services, and add deployment safety like canaries and approval gates."